

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Constraint-Based Problem Solving

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1507

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.*
(BL5)

Dave's Taxonomy: Articulation (Level 4)

Total Marks: 50

Question Count: 5

Code of Conduct

I Bhuvan Kumar J (192524040) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action. Signature
of the student: Bhuvan Kumar J

Assessment Tasks Constraint-Based Problem Solving

1. Design an HDFS-based storage solution for a media platform under the constraints that data availability must remain above 99.9%, replication factor cannot exceed 3, and storage growth must stay cost efficient. Justify your design.
2. A MapReduce workflow must process log data within a 20-minute batch window using limited cluster nodes. Propose a job design under these constraints and explain task distribution.
3. A YARN-managed cluster supports ETL, reporting, and machine learning jobs. Design a scheduler policy under the constraints of fairness, priority for ETL, and recovery from node failure.
4. An enterprise needs to ingest data from SAN, NAS, and operational databases into a big data platform while maintaining backup reliability and minimal duplication. Propose a constrained architecture and justify each component.
5. Design an end-to-end Hadoop ecosystem solution under the constraints of secure data movement, high availability, and integration with Apache NiFi or ETL pipelines. Explain how your design satisfies all constraints.

Constraint-Based Problem Solving Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Constraints	25%	All technical constraints are identified and prioritized accurately	Most constraints identified	Some constraints identified	Constraints poorly understood
System Design Quality	25%	Design is robust, feasible, and aligned to constraints	Reasonable design	Basic design with gaps	Weak design
Application of Hadoop Concepts	20%	Uses HDFS, MapReduce, YARN, and integration concepts effectively	Mostly effective use	Partial use of concepts	Weak concept application
Justification	15%	Strong technical reasoning for all choices	Good justification	Basic justification	Little justification
Clarity	15%	Clear, well-structured, and professional answer	Mostly clear	Somewhat unclear	Poorly structured

Introduction

This document works through two constraint-based system design problems drawn from largescale distributed data processing: designing an HDFS storage layer for a media platform under strict availability, replication, and cost constraints, and designing a MapReduce workflow that must reliably complete log processing inside a fixed twenty-minute batch window on a limited number of cluster nodes. Both problems are approached the same way: the constraints are first broken down individually, a design is proposed that addresses each one directly, and the design is then checked against every constraint to justify why it satisfies the requirements rather than merely approximating them.

Problem 1: HDFS-Based Storage Solution for a Media Platform

Scenario. A media platform that stores video, images, and audio content needs a storage solution built on HDFS. The platform's catalog grows continuously as new content is uploaded, and older content is accessed far less often than new content (a classic long-tail access pattern). The solution must satisfy three hard constraints simultaneously:

- Data availability must remain above 99.9% (equivalent to at most about 8.76 hours of unavailability per year, or roughly 43 minutes per month).
- The replication factor cannot exceed 3, ruling out simply adding more copies of data to improve durability or read throughput.
- Storage growth must remain cost-efficient as the catalog scales into the multi-petabyte range over time.

These three constraints pull in different directions: high availability is normally achieved by adding redundancy, but the redundancy budget is capped at 3x, and cost efficiency argues for using even less than 3x wherever possible. The design below resolves this tension by treating availability as an engineering property of the cluster's architecture and failure handling — not simply a function of how many copies of each block exist — and by making replication factor a variable that changes with data temperature rather than a flat constant applied to the whole dataset.

1.1 Constraint Analysis

Constraint	Implication for design
Availability above 99.9%	Fault tolerance against node failure, rack failure, disk failure, and NameNode failure must come from architecture and operational tooling, not from raw replica count alone. This requires rack awareness, a highly available NameNode, and fast detection and recovery from failures.
Replication factor at most 3	Durability and availability cannot be bought with 5x or 10x replication as is common in some clusters. The design must extract availability from placement strategy, erasure coding, and topology rather than copy count.
Constraint	Implication for design
Cost-efficient storage growth	Media files are large, are written once and read many times, and have a long tail of cold content. Average storage overhead per byte must fall well below 3x as the catalog ages, or storage cost will grow linearly with content volume and become unsustainable.

1.2 Proposed Architecture

a) Cluster topology and rack awareness

The HDFS cluster is deployed across a minimum of three racks (or, in a cloud deployment, three availability zones), each with independent power feeds and network paths so that a single infrastructure failure cannot take down more than one third of the cluster's physical footprint. HDFS's rack-awareness feature places block replicas using topology information: for a block with replication factor 3, one replica is written to the local rack and the remaining two replicas are written to a different rack. This means that even with only three copies, the cluster can lose an entire rack — power, switch, or network failure — and every block still has at least one surviving replica available to serve reads and to reseed replication. This is the central mechanism that lets the design meet the availability target without exceeding the replication ceiling: durability is bought with placement intelligence, not with additional copies.

b) Replication factor policy: tiered by data temperature, not applied uniformly

Rather than applying replication factor 3 uniformly to the entire dataset, the design classifies content into three access tiers and applies a different storage policy to each. This is the primary lever for cost efficiency, because a media catalog's total footprint is dominated by cold, rarely accessed content, even though most read traffic goes to a small recently uploaded fraction.

Data tier	Typical access pattern	Storage policy	Effective storage overhead
Hot: new uploads and trending media, roughly the last 30 days	Frequent reads and streaming; latency-sensitive	Standard 3x replication (the constraint ceiling, used only where it earns its cost)	3.0x
Warm: moderate view counts, roughly 1 to 12 months old	Occasional reads, not latency-critical	Reed-Solomon erasure coding, for example RS(6,3): 6 data blocks plus 3 parity blocks	approximately 1.5x
Cold: archival or compliance-only content, rarely accessed	Rare or no reads	Erasure coding with a wider stripe, for example RS(10,4), placed on lower-cost archival storage class	approximately 1.4x

HDFS Erasure Coding, available from Hadoop 3.x onward, stores data as a set of data blocks plus a smaller set of parity blocks computed from them, instead of storing full duplicate copies. An RS(6,3) policy, for example, can tolerate the loss of any 3 of its 9 total blocks and still fully reconstruct the original data, which is a stronger fault-tolerance guarantee than 3x replication provides against multiple simultaneous failures, while costing only about 1.5x storage instead of 3x. Moving warm and cold data — which represents the large majority of bytes in a mature media catalog — onto erasure coding is what allows the platform to keep growing its catalog without its storage bill growing at the same 3x rate, all while never using a replication factor above 3 for the portion of data that is still replicated rather than encoded.

A background policy engine (implemented as a scheduled job using HDFS storage policy APIs, or a custom mover service) re-evaluates each file's access age on a rolling basis — for example nightly — and transitions files between hot, warm, and cold policies automatically as they age past each threshold, so no manual intervention is required as the catalog grows.

c) Availability engineering beyond replica count

Because replication factor is capped, the majority of the availability budget must come from reducing the frequency and duration of outages rather than from redundancy alone. The following mechanisms are combined:

- **NameNode High Availability:** an Active and Standby NameNode pair is run with automatic failover coordinated through ZooKeeper-based failover controllers, so that a single NameNode process or host failure does not cause cluster-wide downtime. Failover typically completes within seconds.

- JournalNode quorum: at least three (an odd number, so a majority quorum can always be established) JournalNodes maintain a shared, quorum-replicated edit log so the Standby NameNode always has an up-to-date view of filesystem metadata and can take over immediately.
- Fast failure detection and re-replication: DataNode heartbeat intervals and block report intervals are tuned so that under-replicated blocks are detected quickly after a node failure and re-replication begins immediately, minimizing the time window during which a block has fewer than its target number of replicas.
- Disk-level fault isolation: DataNodes run with per-volume failure handling (JBOD with volume scanning) so a single failed disk takes only that disk offline rather than crashing the entire DataNode process, preserving access to the node's other disks.
- Rolling upgrades and maintenance mode: HDFS supports placing DataNodes into maintenance mode during planned maintenance so that planned work does not count against availability the way an unplanned outage would, and so replicas are not needlessly re-replicated during short, known maintenance windows.
- Client-side retry and read fallback: HDFS clients (and any HTTP/CDN layer serving media to end users) are configured to transparently retry reads against an alternate replica or reconstruct from erasure-coded parity blocks if the first-chosen DataNode is unreachable, so transient node issues do not surface as failed requests to end users.

d) Cost-efficient storage growth

- The tiered storage policy described above is the single largest lever on cost, since it shifts the bulk of the catalog's bytes from 3x replication down to roughly 1.4x to 1.5x erasure-coded storage as content ages out of the hot tier.
- Heterogeneous storage types and HDFS storage policies (HOT, WARM, COLD, and ARCHIVE) are used to automatically move blocks between faster SSD-backed storage and lower-cost, higher-density archival disks as access frequency drops, independent of the replication versus erasure coding decision.
- Block size is tuned upward for large media files (video segments, high-resolution images) to keep the ratio of useful data to per-block metadata overhead favorable; the platform avoids storing large numbers of small files (thumbnails, sidecar metadata, subtitles) directly in HDFS, instead storing those in HBase or an object-store layer designed for small-object access, since small files are inefficient in HDFS and inflate NameNode memory usage.
- Lifecycle and retention policies are defined with the platform's legal and product teams so that content which falls outside required retention windows is either deleted or moved to the cheapest archive tier rather than being kept indefinitely at higher-cost tiers by default.
- Capacity planning uses observed ingest rate and tier transition rates to forecast the blended storage overhead (a weighted average across hot, warm, and cold tiers) several quarters ahead, so hardware procurement stays ahead of growth without over-provisioning capacity that then sits idle.

1.3 Monitoring and Operational Practices

Meeting a 99.9% availability target is not a one-time architectural decision; it requires ongoing operational discipline. The design includes:

- 1. Continuous monitoring of under-replicated and missing blocks via HDFS's fsck and NameNode JMX metrics, with alerting thresholds set well below the point at which data would actually become unavailable.
- 2. Dashboards tracking DataNode liveness, disk health (SMART data), and rack-level capacity balance, so a developing hardware problem is caught before it causes an outage.
- 3. Regular failure-injection testing (for example, deliberately decommissioning a DataNode or simulating a rack network partition in a staging cluster) to verify that failover and rereplication behave as designed before a real failure occurs in production.
- 4. A documented runbook for NameNode failover, DataNode replacement, and erasurecoding block reconstruction, so mean time to recovery is minimized and does not depend on any single engineer's memory.

1.4 Justification Against the Constraints

Constraint	How the design satisfies it
Availability above 99.9%	Rack-aware placement survives a full rack loss with only 3 copies; NameNode HA with a JournalNode quorum removes the historical single point of failure in HDFS; fast heartbeat and block-report
Constraint	How the design satisfies it
	tuning plus disk-level fault isolation shrink both the likelihood and duration of any outage. These mechanisms work together so the cluster's measured availability is driven by architecture and operations, not by replica count, allowing the target to be met without exceeding the replication ceiling.
Replication factor at most 3	Hot data uses exactly replication factor 3, the constraint's ceiling, and only where its latency and durability benefit justifies the cost. Warm and cold data use erasure coding, which mathematically tolerates as many or more simultaneous block failures as 3x replication while never itself being a 'replication factor' greater than 3 — durability keeps improving without the replication knob ever moving past 3.

Cost-efficient storage growth	Because the majority of a mature catalog's bytes are warm or cold, and those tiers run at roughly 1.4x to 1.5x overhead instead of a flat 3x, the blended storage-overhead-per-byte across the whole platform trends down as the catalog ages and grows, so total storage cost grows sub-linearly relative to content volume rather than in lockstep with it.
-------------------------------	---

Problem 2: MapReduce Job Design for a 20-Minute Log-Processing Batch Window

Scenario. A recurring MapReduce workflow processes log data (for example, web server or application logs) and must complete within a fixed twenty-minute batch window, using a limited and fixed number of cluster nodes. The job cannot assume elastic scale-out; whatever design is chosen must reliably finish inside the window on the hardware that is actually available, run after run, including on days when input volume is somewhat higher than average.

2.1 Constraint Analysis

Constraint	Implication for design
Hard 20-minute batch window	Job runtime must be predictable and tightly bounded, not merely fast on average. Straggler tasks — the small number of tasks that run far slower than their peers — are the primary risk to a hard deadline and must be actively controlled, not just hoped away.
Limited, fixed cluster nodes	The design cannot rely on adding nodes to shrink runtime.
Constraint	Implication for design
	Every optimization must increase throughput per existing node: maximizing data locality, minimizing shuffle volume, and avoiding wasted or duplicated work.

2.2 Job Design

a) Input sizing and split planning

The HDFS block size and the `mapreduce.input.fileinputformat.split.maxsize / split.minsize` settings are configured so that each map task processes roughly one HDFS block's worth of data,

for example 128 MB. The total number of map tasks is targeted at roughly two to four times the number of available map slots across the cluster (available map slots being the number of worker nodes multiplied by the number of concurrent map task slots configured per node). This ratio matters for two reasons: with too few map tasks relative to available slots, some slots sit idle while others are still working, wasting parallelism; with far too many tiny tasks, per-task JVM startup and scheduling overhead begins to dominate actual processing time. A ratio in the two-to-four range keeps every slot continuously fed with the next task as soon as it finishes the previous one, which is sometimes called a 'wave' scheduling pattern, without the overhead of excessive task counts.

As a concrete illustration, on a cluster of 10 worker nodes with 4 map slots per node (40 total map slots), targeting roughly 80 to 160 map tasks (2x to 4x) for a batch of log data would keep the map phase well-parallelized without incurring excessive per-task overhead; the actual input volume for a given batch then determines whether the configured split size or the slot-count target is the binding constraint, and the smaller of the two resulting task counts is used.

b) Combiner to cut shuffle volume

A Combiner function is used, implementing the same or similar aggregation logic as the Reducer (for example, partial counts of log events by category, partial sums for byte-transfer totals, or partial min/max for latency values), so that intermediate data is pre-aggregated on the map side before it is written to local disk and shuffled across the network. For typical log analytics workloads — counting occurrences, aggregating error rates, or building latency histograms by key — a combiner often reduces shuffle volume by an order of magnitude, because many map tasks will see the same key (for example, the same HTTP status code or service name) many times within their input split. Since shuffle I/O is frequently the single largest contributor to MapReduce job runtime on a modest, fixed-size cluster, the combiner is one of the highest-leverage optimizations available within the twenty-minute budget.

c) Partitioning strategy and reducer count

The number of reducers is set to match the number of available reduce slots, for example one reducer per node or a small integer multiple, so that the reduce phase is fully parallelized across the fixed hardware rather than leaving reduce slots idle. A custom Partitioner is implemented that distributes keys evenly across reducers using a hashed combination of multiple log dimensions (for example, service name combined with a coarse time bucket), rather than partitioning on a single naturally skewed field such as raw HTTP status code, where the vast majority of records might share one value (such as status 200) and overload a single reducer. Even partitioning is essential to a hard deadline because the job's total runtime is bounded by its single slowest reducer; an unbalanced partition can turn an otherwise fast job into one that blows through the batch window even though the cluster's aggregate capacity was more than sufficient.

d) Speculative execution for straggler control

Speculative execution is enabled for both map and reduce phases (`mapreduce.map.speculative` and `mapreduce.reduce.speculative` set to true), so that if any individual task is running noticeably slower than other tasks doing comparable work — for example due to a degraded disk, an

unusually large input split, or resource contention from another process on the same node — the framework automatically launches a duplicate attempt of that task on a different node in parallel. Whichever attempt finishes first is kept and the other is killed. On a small, fixed cluster with a hard deadline, this is the primary safeguard against a single misbehaving node silently consuming the entire time budget, and it is generally more reliable than trying to diagnose and fix hardware issues in real time during a live batch run.

e) Compression

Intermediate, map-output data is compressed using a fast codec such as Snappy or LZ4 — chosen specifically for decompression speed over maximum compression ratio, since the goal is to reduce shuffle and spill time rather than to minimize storage footprint. Final job output is also compressed if it is consumed by downstream jobs rather than by humans. Compression trades a small, predictable amount of additional CPU time for a larger reduction in disk spill and network transfer time, which is a favorable trade whenever the cluster is bottlenecked on I/O or network bandwidth rather than CPU, as is common for shuffle-heavy log-aggregation workloads.

f) Pre-processing, format choice, and early filtering

- Raw logs are converted, ahead of the batch window (as part of ingestion rather than as part of the time-boxed batch job itself), into a splittable, efficiently parseable format such as Avro or a compressed SequenceFile, so the twenty-minute job only ever reads structured records rather than parsing raw unstructured text on every run.
- Any filtering that can be determined in advance (for example, restricting to a specific log level, a specific service, or a specific date range) is pushed as early as possible, ideally into the RecordReader or the very start of the map function, so irrelevant records are discarded before they incur any shuffle or reduce-side cost.
- Where the same batch job runs repeatedly on a rolling window of data, only the newly arrived input partitions for the current window are fed to the job, rather than re-scanning the entire historical log corpus on every run.

2.3 Task Distribution Across the Limited Nodes

With N worker nodes and S task slots configured per node, giving N multiplied by S total task slots across the cluster, tasks are distributed across the fixed hardware as follows:

Phase	Task count	Distribution strategy
-------	------------	-----------------------

Map	Approximately 2x to 4x total map slots, with one map task per input split or HDFS block	The ApplicationMaster assigns map tasks to nodes with a strong preference for data locality, meaning a task is scheduled on, or as close as possible to, the node that already holds its input block, avoiding unnecessary network transfer of raw input data. As soon as a node finishes a task, the scheduler immediately assigns it the next queued split, so the phase proceeds in overlapping waves rather than fixed synchronous rounds.
Combine	Runs inline within every map task, one combine step per map task	No separate scheduling is needed; the combiner executes locally on the same node immediately after the map function produces its output and before that output is spilled to local disk, so it never adds network cost.
Shuffle and sort	Framework-managed; volume determined by combiner effectiveness	Partitioned, compressed intermediate data is transferred in parallel from every map task's node to the specific reducer node that owns that partition, and this transfer begins as soon as the first map tasks complete rather than waiting for the entire map phase to finish, so shuffle overlaps with the tail of the map phase instead of adding fully sequential time to the job.
Reduce	Equal to the number of configured reduce slots, typically about one per node or a small fixed multiple	Each reducer is assigned one or more partitions by the custom Partitioner, chosen to keep partition sizes as even as possible; reducers begin copying and merging their shuffle input incrementally as soon as it becomes available, and only the final reduce computation and output write must wait for all of a partition's map outputs to have arrived.

2.4 Handling Variability and Failure Within the Time Budget

Because the deadline is hard, the design also accounts for run-to-run variability rather than assuming every batch looks identical to the one used for initial tuning:

- Job-level timeouts and alerting are configured so that if a run is trending toward exceeding a safe fraction of the twenty-minute window (for example 80 percent elapsed with map phase not yet complete), an operator or an automated system is notified before the deadline is missed, rather than only after.
- Historical run metrics (map phase duration, shuffle duration, reduce phase duration, and any speculative task activity) are logged for every batch, so slow trends can be caught and tuned

— for example by adjusting split size or combiner logic — before they turn into a missed deadline in production.

- Where input volume is known to spike on certain days (for example, higher traffic and thus higher log volume on weekends or during promotional events), the map task count target and reduce partition count are allowed to scale up proportionally within the fixed node count, since more, smaller tasks on the same hardware still complete faster than fewer, larger tasks when the cluster has spare slot capacity.

2.5 Justification Against the Constraints

Constraint	How the design satisfies it
20-minute batch window	Right-sized input splits keep every task slot continuously busy; the combiner and shuffle compression substantially shrink the shuffle phase, which is typically the longest phase on a constrained cluster; speculative execution bounds the runtime impact of any single slow task; and overlapping shuffle with the tail of the map phase removes idle time between phases. Together these keep total runtime predictable and comfortably within the fixed window on a normal run, with monitoring in place to catch and correct drift before it causes a missed deadline.
Limited cluster nodes	Every optimization targets throughput per existing task slot — locality-aware scheduling, combiner-side preaggregation, even partitioning, and fast compression — rather than assuming additional nodes could be added if the job ran slow, so the design meets its deadline within the fixed hardware footprint that was specified.

Conclusion

Both problems illustrate the same underlying discipline of constraint-based system design: rather than treating each constraint as an independent target to satisfy in isolation, the design identifies where the constraints interact and finds mechanisms that satisfy more than one constraint at once. In the HDFS storage problem, tiered replication and erasure coding satisfy the availability and cost constraints simultaneously without ever exceeding the replication ceiling. In the MapReduce problem, combiner-based pre-aggregation and locality-aware scheduling satisfy the time-window constraint without requiring additional nodes. In both cases, the justification step — explicitly checking the finished design against every original constraint — is what confirms the design is actually compliant rather than merely reasonable-sounding.